Lecture 20

Backend Security

Backend Engineering from First Principles

Detailed study notes • injection • authentication • authorization • XSS/CSRF • secure configuration

Lecture 20 — Backend Security

Backend Engineering from First Principles

> *Source basis: the supplied lecture transcript (approximately 2 hours 50 minutes). These notes preserve the lecture's main mental models, examples and terminology. Short practical clarifications are included where they help interpret the concepts safely.*

1. Why backend security matters

Security is one of the most consequential responsibilities in backend engineering. A security failure can cause financial loss, expose private data, damage customer trust and consume engineering and operational resources.

Security is a very large domain. The lecture distinguishes several contexts:

- **Browser security:** HTML, JavaScript, cookies, local storage and browser-side vulnerabilities.
- **Network security:** HTTP/HTTPS, encryption and communication between systems.
- **Server/OS security:** the operating system and process environment where the backend runs.
- **Backend application security:** vulnerabilities caused by the application's own code and design.

The lecture deliberately focuses on practical backend application security rather than attempting to cover every security discipline.

Core mindset: No application can be guaranteed to be perfectly secure forever. Technology, libraries and attack techniques evolve. The goal is to develop a habit of questioning assumptions whenever code is written.

2. Think like an attacker: question developer assumptions

The lecture's central question is:

“Where did the developer make an assumption?”

Common unsafe assumptions include:

- User input will always be clean and correctly formatted.
- A user is who they claim to be.
- Requests reaching the backend must have come from the intended frontend.
- Users will only click the buttons and follow the workflows the developer designed.
- Nobody will inspect browser network calls or modify request parameters.
- IDs, roles, prices, permissions and other values sent by the client can be trusted.

Attackers do not use an application only through its intended UI. They inspect requests, alter parameters, replay calls and probe boundaries.

A useful habit is to ask, for every feature:

1. What data crosses into this component?
2. Who controls that data?
3. Which system or language will interpret it?

4. What assumptions am I making?
5. What happens if the input or identity is malicious or unexpected?

3. Injection attacks: code versus data

3.1 Why injection happens

A backend interacts with several systems and therefore several languages:

- SQL when communicating with a relational database.
- Shell/OS command syntax when invoking system tools.
- HTML and JavaScript when producing browser-rendered content.
- Other formats/protocols depending on the application, such as XML or LDAP.

Each language has grammar, operators and special characters. A character that is ordinary text in one context may have command meaning in another.

Injection vulnerabilities arise when data supplied by a user crosses into another language's context and is interpreted as part of that language's executable structure.

Root cause: confusion between code and data. User-controlled data is accidentally treated as executable instructions.

3.2 SQL injection

The lecture demonstrates a login-style query that searches for a user by email.

Unsafe pattern:

```
-- Conceptual unsafe pattern: do not build queries this way
SELECT * FROM users WHERE email = '<user_input>';
```

If the backend constructs the query by concatenating a user-provided string into the SQL text, an attacker may insert quote characters, Boolean expressions, comment markers or statement separators.

The lecture's example uses a condition equivalent to `' OR '1'='1' --`:

- The leading quote can close the string opened by the application.
- `OR` adds another condition.
- `1 = 1` is always true.
- `--` comments out the remaining SQL on that line.

The intended filter can therefore be changed into a condition that matches every row. The lecture also demonstrates a more destructive hypothetical payload using a statement separator and `DROP TABLE`.

Potential consequences include:

- Reading records the user should not see.
- Extracting sensitive information from other tables using techniques such as `UNION`.
- Modifying or deleting data.
- In some database/configuration combinations, reaching files or OS-level capabilities.

The exact impact depends on the database, driver, permissions and configuration.

3.3 Prevent SQL injection with parameterized queries

Use **parameterized queries**, also called **prepared statements**.

```
SELECT * FROM users WHERE email = $1;
```

Then pass the user value separately as a parameter through the database driver's query/execute API.

The key distinction:

- Query structure is supplied as SQL code.
- User input is supplied separately as a value.
- The database driver treats the parameter as data, not as SQL syntax.

Do not concatenate or interpolate user input into SQL strings.

Validation is still useful: an email field should be validated as an email. But validation is not a replacement for parameterized queries. If validation has a gap, parameterization still protects the SQL structure.

3.4 Database permissions as defense in depth

The lecture recommends limiting the database account used by the backend to the permissions it needs. It discusses granting data manipulation permissions such as `INSERT`, `UPDATE` and `DELETE`, while avoiding unnecessary data-definition permissions such as changing or dropping tables.

Practical principle: apply least privilege. A compromised query should not automatically have every permission available to a database administrator.

3.5 NoSQL injection

Using a NoSQL database does not remove injection risk.

The lecture uses MongoDB-style JSON query objects:

- A normal query may compare an `email` field with a string.
- Query objects can also contain operators, often represented by special keys beginning with `$`.
- If an application accepts an entire user-controlled JSON object and passes it directly into a database query, the user may be able to control query operators and structure.

Do not let untrusted users supply arbitrary database query objects. Construct query structure on the server and accept only the specific validated values the endpoint requires.

4. OS command injection

Command injection has the same root cause as SQL injection, but the target is the operating system or a command-line program.

4.1 Example from the lecture

Imagine a backend that accepts an uploaded image and uses a command-line tool such as FFmpeg to resize or compress it. The backend constructs a command and inserts a user-controlled filename into that command.

If the application builds one shell command string by concatenating user input, shell metacharacters such as semicolons, pipes or ampersands may change what gets executed.

The lecture illustrates how a malicious filename could append another command to the intended image-processing command.

4.2 Prevention

Separate the executable from its arguments:

- Use the language's process-spawning API that accepts an executable and an argument array/list separately.
- Avoid invoking a shell interpreter when it is not required.
- Do not build shell command strings by concatenating user input.
- Validate filenames and constrain file paths to an intended directory.
- Run processing tools with minimal OS permissions.

General injection rule: use APIs that keep command structure separate from user-controlled values.

5. Authentication security

Authentication answers: **Who is this user?** It verifies the identity associated with the request.

If authentication fails, an attacker may impersonate a user, access private data, act on the user's behalf or steal money.

5.1 Consider a dedicated identity provider

The lecture recommends considering a third-party authentication/authorization provider for production systems when budget and organizational approval allow it.

Reasons:

- Authentication is broader than verifying a JWT.
- Production systems may need session/device management, revocation, social login, account linking, permission management and many edge cases.
- Implementing and maintaining these flows consumes engineering time.
- A specialist provider focuses its engineering and security effort on identity systems.

The lecture names providers such as Clerk as examples. This is a recommendation from the lecture, not a claim that every provider is automatically secure or suitable.

Even when using a provider, engineers must understand the flows, configure them correctly and integrate them securely.

5.2 Stateful authentication and session revocation

The lecture describes stateful authentication as a system that can track active sessions/devices and revoke them. For example, a user or service operator may log out all devices.

This capability can involve:

- Persistent session records.
- Databases and caches.
- Session expiration and timeout policies.
- Revocation and invalidation across storage layers.

- Coordinating session state with authentication requests.

Stateful authentication offers control over active sessions, but introduces storage and consistency considerations.

5.3 Social login and account linking

Social login includes options such as “Sign in with Google” or “Sign in with GitHub.” The lecture highlights account linking as an important edge case.

Example:

1. A user initially creates an account using email and password.
2. Later, the same person chooses Google sign-in with the same email address.
3. The identity system must correctly associate the external identity with the existing account rather than accidentally creating an unrelated duplicate account.

OAuth-based social login involves redirects and coordinated frontend/backend behavior. Do not treat it as merely adding a button.

6. Password storage

6.1 Never store plaintext passwords

The naive approach is to store the exact password in a database column. The lecture explains why this is dangerous:

- Database breaches happen.
- Anyone with sufficient database access could read users' passwords.
- Users often reuse passwords across websites, so one leaked password can lead to account takeover elsewhere.
- Internal employees or contractors with database access should not be able to read users' original passwords.

6.2 Hash passwords

A hash function maps input data to a fixed-length output. Password storage uses a one-way password-hashing process so the original password is not stored.

At login:

1. User submits a password.
2. Backend retrieves the stored password hash and its associated parameters.
3. A password-hashing verification function checks the submitted password against the stored hash.
4. The application allows or denies login based on the result.

Do not decrypt a stored password; a correctly designed password-storage scheme is based on one-way verification.

Practical clarification: For passwords, use a purpose-built, deliberately slow password-hashing algorithm such as Argon2id, bcrypt or scrypt (or an appropriately configured PBKDF2 implementation). Do not use a fast general-purpose hash such as plain SHA-256 for password storage. Use a unique salt per password; modern password-hashing libraries usually manage this.

7. JWTs: structure, lifecycle and risks

The transcript discusses JWTs in the context of authentication and authorization vulnerabilities.

7.1 JWT structure

A JSON Web Token commonly has three dot-separated parts:

1. **Header:** metadata such as the signing algorithm and token type.
2. **Payload:** claims, such as subject/user ID, issue time, expiry and optional custom claims.
3. **Signature:** allows the server to verify that the signed header and payload have not been altered.

The lecture emphasizes that the payload is Base64URL-encoded, not encrypted. Anyone who obtains the token can decode and read its payload.

Do not put passwords, secrets or confidential personal data in an ordinary signed JWT payload.

7.2 Typical request flow

1. Server creates a token with claims.
2. Server signs the token.
3. Client stores it using an appropriate secure mechanism.
4. Client sends it with subsequent requests.
5. Server verifies the signature and validates relevant claims, including expiry and issuer/audience where applicable.
6. Server uses the verified identity/claims as part of request authentication and authorization.

A valid signature means the token has not been modified by someone who lacks the signing key. It does not, by itself, prove that the token is still authorized or unexpired.

7.3 JWT advantages and limitations

The lecture notes that JWTs can make scaling easier because a service can verify a token without looking up a central session record for every request.

But JWTs have a revocation problem:

- A signed token can remain valid until its expiry.
- Deleting a copy in the browser does not invalidate a stolen copy.
- Logging out one device or all devices immediately is difficult without additional server-side state.

7.4 Workarounds discussed in the lecture

Token blacklist: keep server-side records of tokens that have been revoked. This introduces a lookup/state requirement.

Access token + refresh token:

- Access token is short-lived and sent with normal API requests.
- When it expires, the client uses a refresh token to obtain a new access token.
- Refresh tokens are longer-lived and should be managed with server-side controls, rotation/revocation and secure storage.

- A stolen refresh token remains a serious risk, so its lifecycle must be protected.

The lecture's general recommendation is to use short-lived access tokens (minutes to hours rather than days) and a refresh-token flow with server-side storage when JWTs are used.

7.5 Where should a browser store a token?

The lecture compares local storage and cookies:

- **Local storage:** accessible to JavaScript running on the page. If the site has an XSS vulnerability, malicious JavaScript may read and exfiltrate a token stored there.
- **Cookies:** can be configured with security attributes that reduce exposure.

The lecture recommends HTTP-only cookies rather than local storage for browser token storage.

Practical cookie attributes to understand:

- `HttpOnly`: JavaScript cannot read the cookie through `document.cookie`.
- `Secure`: cookie is sent only over HTTPS.
- `SameSite`: controls when cookies are sent in cross-site contexts and affects CSRF risk.

HTTP-only cookies reduce token theft through JavaScript, but do not eliminate XSS or all session risks.

8. Rate limiting and brute-force protection

A backend exposed to the internet can receive a large number of requests. Attackers may try to overwhelm a service or repeatedly guess credentials.

Rate limiting restricts how frequently a client or identity can perform an action during a time window.

The lecture discusses several approaches:

8.1 Per-IP rate limiting

Limit requests based on source IP address.

Limitations:

- Many users in a university, office or organization may share one public IP through NAT.
- VPNs, proxies, botnets and rotating IP addresses can distribute an attack across many IPs.
- IP addresses are not always a reliable representation of one human or account.

8.2 Per-account limiting or lockout

Limit failed login attempts for a specific account. The lecture gives an illustrative policy of a small number of failed attempts in a time window followed by a temporary lockout.

This can help when an attacker rotates IP addresses but targets one account.

However, attackers can distribute attempts across many accounts, making per-account limits alone insufficient. Aggressive lockout policies can also be abused to deny service to legitimate users.

8.3 Layered rate limiting

The lecture's examples point toward combining multiple signals:

- Per-IP limits.

- Per-account or per-identity limits.
- Endpoint-specific limits.
- Monitoring and alerting for suspicious patterns.
- Temporary blocks or additional verification when abuse is detected.

Use rate limiting as a layered control, not as a single perfect defense. Configure trusted proxy handling carefully before relying on forwarded IP headers.

9. Authorization vulnerabilities: BOLA and BFLA

Authentication tells the system who the user is. Authorization determines what that user is allowed to do.

The lecture stresses that authorization must be checked at the point where the protected data or function is accessed.

9.1 Broken Object Level Authorization (BOLA)

BOLA occurs when an application checks that a user is logged in but fails to verify that the user is allowed to access the specific object they requested.

Example:

- User A is logged in and can access their own books or invoices.
- The frontend calls an endpoint with `id=5`.
- An attacker changes the ID to `id=6`.
- If the backend fetches object 6 without checking ownership/permission, User A may see User B's data.

The ID may be in a URL path, query parameter or request body. Hiding an ID in the frontend is not authorization.

Safer pattern: include the authenticated user's identity in the data-access condition.

```
SELECT *
FROM invoices
WHERE id = $1
AND user_id = $2;
```

Here `$2` must come from the server's verified authentication context, not from a user-controlled field.

The lecture also discusses returning `404 Not Found` when the object is not available to the caller, rather than revealing whether another user's object exists. This can reduce information leakage.

9.2 Broken Function Level Authorization (BFLA)

BFLA occurs when a user can invoke a function or endpoint that should be restricted to another role or permission level.

Example:

- A normal user discovers or manually calls an admin endpoint such as `/admin/invoices`.
- If the backend only checks that the user is logged in, the user may access an administrative function.

Never rely on a hidden URL, frontend navigation or button visibility as a security control. “Security through obscurity” is not authorization.

Enforce role/permission checks on the server for every protected operation.

9.3 Role-based and permission-based checks

The lecture distinguishes roles and granular permissions:

- A role might be `admin` or `user`.
- Permissions might specify `read invoices`, `write invoices` or access to a particular entity.
- Two users can have the same permission but different roles, or a role can grant a defined set of permissions.

For robust systems, define a clear authorization layer and consistently apply it across endpoints and data access.

9.4 IDOR and horizontal authorization

The lecture refers to IDOR (Insecure Direct Object Reference) in connection with changing object identifiers to access another user's data.

It also describes **horizontal authorization**: one ordinary user accesses another ordinary user's resources. This differs from vertical privilege escalation, where a lower-privileged user gains access to a higher-privileged function or role.

Random or UUID identifiers can make guessing harder, but they do not replace authorization checks. An unguessable ID is not proof of permission.

9.5 Authorization checklist

- Derive user identity from verified server-side authentication context.
- Check access to the specific object, not merely the endpoint.
- Check permissions for privileged functions.
- Apply checks consistently in service/repository access paths.
- Do not trust IDs, roles or permissions supplied by the client.
- Avoid revealing whether another user's private object exists when that disclosure is unnecessary.
- Log and investigate suspicious cross-account access attempts.

10. Cross-Site Scripting (XSS)

XSS occurs when an attacker causes their JavaScript to execute in another user's browser in the context of a trusted website.

Possible impacts include:

- Reading or acting on data available to the page.
- Making authenticated API requests as the victim.
- Stealing tokens that are accessible to JavaScript.
- Redirecting users to phishing pages.
- Modifying page content or user interactions.

10.1 Stored XSS

The lecture's example is a comment or Markdown feature:

1. User submits Markdown.
2. Backend stores the submitted content.
3. Application converts Markdown to HTML using a renderer.
4. Frontend injects the resulting HTML into the DOM.
5. If malicious HTML/JavaScript is allowed through, it may execute for every user who views the comment.

The lecture mentions frontend rendering libraries in the Markdown ecosystem and React's `dangerouslySetInnerHTML` as a point requiring care.

Key risk: content that was intended as data becomes executable HTML/JavaScript.

10.2 Reflected XSS and DOM-based XSS

The lecture also names:

- **Reflected XSS:** attacker-controlled input is reflected into a response and rendered in a way that executes script.
- **DOM-based XSS:** unsafe client-side JavaScript uses attacker-controlled data to modify the DOM in an executable context.

10.3 XSS prevention and defense in depth

- Escape output for the context in which it is rendered.
- Sanitize user-authored HTML/Markdown with a well-maintained allowlist-based sanitizer when HTML rendering is required.
- Avoid unsafe HTML injection APIs where possible.
- Do not assume input validation alone can make arbitrary HTML safe.
- Use a restrictive **Content Security Policy (CSP)** as an additional layer.

CSP is delivered through an HTTP response header and can restrict script sources and other resources. A policy that blocks inline scripts can reduce the impact of certain injected scripts.

Important: CSP is defense in depth, not a substitute for safe output handling and sanitization. An overly permissive policy may provide little protection.

11. Cross-Site Request Forgery (CSRF)

CSRF exploits the browser's behavior of automatically attaching cookies to requests.

High-level scenario:

1. User is logged in to a trusted website and has an authenticated cookie.
2. User visits an attacker-controlled website in another tab.
3. The attacker's page causes the browser to send a request to the trusted website.
4. If the browser automatically attaches the authentication cookie and the server does not verify request intent, an unwanted action may occur as the logged-in user.

The lecture says modern browser defaults and frameworks reduce some CSRF exposure, but developers should understand the mechanism.

Defenses discussed

- Configure cookie `SameSite` appropriately. The lecture notes that `Lax` is a common default and that `None` changes cross-site cookie behavior.
- Use CSRF tokens or framework-provided CSRF protections where appropriate.
- Validate request origin/referer as an additional signal where suitable.
- Do not treat CORS as a CSRF defense.

Clarification: CORS controls whether browser JavaScript can read cross-origin responses; it does not generally stop a browser from sending every kind of cross-origin request. Cookie-based state-changing endpoints still need appropriate CSRF defenses.

12. CORS: what it does and does not do

The lecture mentions CORS in the CSRF discussion and explicitly warns that CORS is not itself a security mechanism for proving that a request came from the legitimate frontend.

CORS is a browser-enforced policy that controls whether frontend JavaScript from one origin may access responses from another origin. It is not authentication, authorization or a substitute for backend validation.

Do not assume a request is trustworthy merely because it carries an `Origin` header or appears to come from your frontend. Non-browser clients can construct HTTP requests too.

13. Security misconfiguration and secrets management

13.1 Secrets must not be committed to source control

Secrets include:

- API keys.
- Database passwords.
- Encryption keys.
- JWT signing secrets.
- Authentication-provider secrets and other credentials.

A common mistake is hard-coding a secret in source code and committing it to Git. Once pushed to GitHub, GitLab or Bitbucket, deleting the visible line does not necessarily remove it from commit history.

Recommended approach:

- Keep secrets out of source code and version control.
- Use environment variables or a dedicated secret-management system.
- Restrict access to secrets using least privilege.
- Rotate/revoke a secret immediately if it is exposed.
- Review repository history and logs for accidental disclosure.

The lecture names services such as AWS Parameter Store and HashiCorp Vault as examples of secret/configuration management systems.

13.2 Debug mode in production

Debug mode is useful during development but dangerous in a live environment. Verbose errors and stack traces may reveal:

- Internal code paths.
- Database queries and schema details.
- File paths and infrastructure information.
- Framework/runtime details.
- Other information that helps an attacker understand the system.

Use production-safe error responses and keep detailed diagnostics in access-controlled logs. Do not expose stack traces or raw internal errors to clients.

13.3 Security headers and middleware

The lecture mentions modern middleware and security-related HTTP headers as useful layers of defense. It specifically references CSP and headers that help protect against issues such as clickjacking.

Examples of headers to understand:

- `Content-Security-Policy` (CSP).
- `X-Content-Type-Options: nosniff`.
- `Strict-Transport-Security` (HSTS), when HTTPS is correctly deployed.
- `X-Frame-Options` or CSP `frame-ancestors` for framing/clickjacking controls.
- `Referrer-Policy`.

Headers should be configured deliberately for the application; blindly copying policies can break functionality or create a false sense of security.

14. The lecture's layered security model

The closing message of the lecture is that security failures often happen when data crosses a boundary and the developer makes an unsafe assumption.

The examples map to boundaries:

- User input crossing into SQL.
- User input crossing into shell command syntax.
- Markdown or HTML crossing into the browser DOM.
- A request crossing the boundary of what one user is allowed to access.
- A browser request crossing origins while cookies carry identity.

A useful layered approach:

1. **Validate inputs:** check type, shape, size, allowed values and business constraints at the boundary.

2. **Separate data from executable structure:** use parameterized SQL, argument arrays and safe rendering APIs.
3. **Authorize at the point of access:** verify object ownership and function permissions server-side.
4. **Limit blast radius:** least-privilege database and OS accounts, restricted secrets and minimal permissions.
5. **Add defense-in-depth controls:** CSP, secure cookie attributes, security headers, rate limiting and monitoring.
6. **Keep sensitive diagnostics private:** safe client errors and protected logs.
7. **Keep learning:** review current vulnerability guidance and test assumptions continuously.

No single layer is perfect. Multiple independent layers reduce the chance that one missed check becomes a catastrophic incident.

15. Security resources named in the lecture

The lecture recommends further independent study and specifically mentions:

- **PortSwigger Web Security Academy:** practical web-security learning and labs, including injection, authentication, authorization and XSS topics.
- **OWASP Top 10:** a periodically updated awareness list of major web application security risk categories.
- **OWASP Cheat Sheet Series:** practical guidance for topics such as authentication and session management.

The lecture explicitly says its coverage is not comprehensive and encourages deeper reading.

16. Practical backend security checklist

Input and injection

- ■ Treat every client-supplied value as untrusted.
- ■ Validate types, formats, ranges, lengths and allowed values.
- ■ Use parameterized SQL queries.
- ■ Build NoSQL query objects on the server; do not accept arbitrary client query structures.
- ■ Use process APIs with separate executable and argument arrays.
- ■ Avoid unsafe shell-string construction.
- ■ Escape/sanitize output for the exact HTML/JavaScript context.

Identity and sessions

- ■ Use a well-maintained authentication solution where appropriate.
- ■ Never store plaintext passwords.
- ■ Use a purpose-built password-hashing algorithm.
- ■ Validate JWT signature, issuer, audience, expiry and intended use.
- ■ Keep JWT payloads free of secrets.

- Use short-lived access tokens and a secure refresh-token strategy where applicable.
- Protect browser cookies with `HttpOnly`, `Secure` and suitable `SameSite` settings.

Authorization

- Authenticate every protected request.
- Check object-level access for every requested record.
- Check function/role permissions for privileged endpoints.
- Never rely on hidden URLs or frontend-only checks.
- Use server-derived identity in database filters.

Abuse and configuration

- Apply layered rate limits to sensitive endpoints.
- Monitor suspicious login and access patterns.
- Keep secrets out of Git and rotate exposed credentials.
- Disable debug mode and verbose public errors in production.
- Configure CSP and other security headers deliberately.
- Use least-privilege service/database/OS accounts.
- Keep dependencies and security configuration maintained.

17. Interview questions and answers

Q1. What is the root cause of injection attacks?

A failure to separate code from data. Untrusted input is interpreted as part of another system's executable language.

Q2. How does SQL injection work?

Unsafe query construction concatenates user input into SQL. Special characters and operators in the input can change the query's meaning.

Q3. How do parameterized queries prevent SQL injection?

They keep query structure separate from values. The database driver sends user values as data rather than parsing them as SQL syntax.

Q4. Does using MongoDB eliminate injection?

No. If an application lets users control query-object structure or operators, NoSQL injection may occur.

Q5. What is command injection?

It occurs when user-controlled data is inserted into an OS command or shell string and interpreted as additional command syntax.

Q6. What is the difference between authentication and authorization?

Authentication verifies who the user is. Authorization decides which resources and operations that identity may access.

Q7. Why should passwords be hashed?

A database breach should not reveal the original passwords. Password hashing makes stored values unsuitable for direct recovery and verification uses a password-hashing function.

Q8. Why is plaintext password storage dangerous?

A database breach or inappropriate internal access exposes the actual passwords, which users may have reused on other services.

Q9. What are the three parts of a JWT?

Header, payload and signature.

Q10. Is a JWT payload encrypted?

Not by default. It is Base64URL-encoded and can be decoded. A signature protects integrity, not confidentiality.

Q11. What is the JWT revocation problem?

A signed token can remain usable until expiry unless the system adds server-side revocation/state or another control.

Q12. Why are short-lived access tokens useful?

They reduce the time window in which a stolen access token can be reused. Refresh-token handling is still security-sensitive.

Q13. What is BOLA?

Broken Object Level Authorization: a user can access an object belonging to another user because the backend fails to check authorization for that specific object.

Q14. What is BFLA?

Broken Function Level Authorization: a user can invoke a function or endpoint that should be restricted to a different role or permission level.

Q15. Why are UUIDs not a replacement for authorization?

Hard-to-guess identifiers reduce enumeration but do not prove that the caller has permission to access the resource.

Q16. What is XSS?

An attacker causes JavaScript to execute in another user's browser in the context of a trusted site.

Q17. What is the difference between stored and reflected XSS?

Stored XSS persists malicious content and executes when users view it. Reflected XSS is returned in a response based on a request and rendered unsafely.

Q18. What is CSP?

Content Security Policy is a browser-enforced policy, usually delivered through an HTTP header, that restricts sources and types of content the page may load or execute.

Q19. What is CSRF?

An attacker tricks a browser that has an authenticated session into sending an unwanted request to the trusted site.

Q20. Is CORS a replacement for CSRF protection?

No. CORS governs browser access to cross-origin responses; it is not a general guarantee that cross-origin requests cannot be sent.

Q21. Why is debug mode dangerous in production?

It can expose stack traces, SQL, file paths and internal implementation details that help attackers.

Q22. What should happen if a secret is committed to Git?

Treat it as compromised, revoke/rotate it, investigate use and remove it from active code/history as appropriate. Deleting the latest visible line alone is insufficient.

18. Quick revision sheet

- **Security mindset:** question every developer assumption.
- **Boundary:** where data moves from one system/language/context to another.
- **Injection:** code and data become confused.
- **SQL injection fix:** parameterized queries/prepared statements.
- **NoSQL injection:** do not accept arbitrary client-controlled query structure.
- **Command injection fix:** executable and arguments passed separately; avoid shell strings.
- **Authentication:** who are you?
- **Authorization:** what can you access/do?
- **Password storage:** purpose-built one-way password hashing; never plaintext.
- **JWT:** header + payload + signature; payload is readable, not encrypted by default.
- **JWT revocation:** needs extra state/control; short access-token lifetime helps.
- **BOLA:** unauthorized access to another user's object.
- **BFLA:** unauthorized access to a protected function.
- **UUIDs:** reduce guessing, do not grant permission.
- **XSS:** attacker script executes in trusted site context.
- **CSP:** useful defense in depth, not a substitute for safe rendering.
- **CSRF:** browser sends authenticated cookies with an attacker-induced request.
- **CORS:** browser response-sharing policy, not authentication or general CSRF protection.
- **Secrets:** never commit; use a secret manager and rotate leaks.
- **Production:** disable debug mode and protect stack traces.
- **Final principle:** validate inputs, separate code/data, authorize at access, limit privileges and layer defenses.